Newton 1.2 Solver Cold-Start Compile Benchmark

Hardware: RTX 5000 Ada Generation Laptop (sm_89, 16 GB), Windows 11. **Software:** Newton 1.2.0, Warp 1.13.0, CUDA driver 13.0, mujoco-warp 3.8.1. **Date:** 2026-05-13.

TL;DR

Newton 1.2's three GPU solvers differ by **~2.3×** in cold-start compile cost. MuJoCo (~142 s) is the heaviest, Kamino (~133 s) close behind, VBD (~62 s) the lightest. **Once cached, every solver starts in 5–8 s.** Cold cost is dominated by the `Example.__init__` phase, which compiles all Warp/NVRTC kernels via the example's warm-start `solver.step()` call.

Methodology

For each solver: pick one representative example, run cold + warm, headless (`--viewer null`, `--num-frames 1`). Each iteration deletes **both** the Warp source-hash cache (`%LOCALAPPDATA%\NVIDIA\warp\Cache\1.13.0`) and the CUDA driver's NVRTC ComputeCache (`%APPDATA%\NVIDIA\ComputeCache`) before the cold run. Solver order is Fisher–Yates shuffled per iteration to spread OS file-cache warmth. **3 iterations.** Phase timings come from a Python instrumentation wrapper (`time_example_phases.py`) that drives each example's `Example` class directly and exits cleanly after one frame; this avoids ViewerGL's blocking run loop.

Solver → example mapping

Solver	Example	Scenario
Kamino (PADMM)	`kamino_robot_dr_legs`	Disney quadruped, 36 joints, 1 world
MuJoCo (mujoco-warp)	`robot_anymal_d`	ANYmal D quadruped, 1 world
VBD	`cloth_hanging`	64 × 32 cloth on ground plane

Roll-up (totals, seconds)

Kind	Kamino	MuJoCo	VBD
Cold (median)	**132.5**	**142.4**	**61.9**
Cold (range)	126.3 – 136.2	139.5 – 177.9	60.2 – 62.0
Warm (median)	**7.9**	7.3	**4.9**
Warm (range)	7.6 – 48.5†	7.1 – 8.0	4.9 – 4.9

†One 48.5 s warm outlier on Kamino: Python import phase ballooned to 37 s, likely because the preceding cold run rewrote ~600 MB of NVRTC cache and evicted Python source pages from OS file cache. Solver init was a normal 9.9 s.

Where the cold time goes (median, seconds)

Phase	Kamino	MuJoCo	VBD
`import newton` + module	3.4	3.5	3.5
Viewer ctor (Null)	0.2	0.2	0.2
`Example.__init__` (USD + model + **Warp compile**)	**129.0**	**137.9**	**51.1**
1 step (CUDA graph capture'd)	0.0	0.0	0.0
1 render	0.0	0.0	**7.0**

For Kamino and MuJoCo, ~97 % of cold time is in `Example.__init__`, specifically the warm-start `solver.step()` that triggers JIT compile of every kernel the solver will need. **VBD is the odd one:** ~14 % of its cold cost (7 s) lands in the **first render** because the deformable visualization compiles its own kernels lazily on first draw rather than during init.

Per-solver kernel hot-spots (cold, single-iteration trace)

Kamino — 38 modules, ~75 s compile. Two kernels are 59 % of the cost: - `kamino.linalg.factorize.llt_blocked` — **26.0 s** (blocked sparse Cholesky) - `kamino.solvers.fk.kernels` — **18.4 s** (forward kinematics) - PADMM kernels — 4 instantiations × ~3 s each - Bucket: 65.5 s solver, 3.3 s viewer, 2.2 s geometry, 2.2 s narrowphase

MuJoCo — 49 modules, ~110 s compile. Broad and shallow, no single dominant kernel: - `newton.sim.articulation` — 12.5 s - `newton.solvers.mujoco.kernels` — 12.4 s - `update_gradient_cholesky` — 11.2 s - `tile_cholesky_factorize` — 10.5 s - `mesh_triangle_contacts_to_reducer` — 10.3 s - Bucket: 28.1 s across `mujoco_warp.*` submodules (sensor/smooth/forward/io/constraint/solver/passive)

VBD — 14 modules, ~53 s compile. Smallest count, evenly distributed: - `vbd.rigid_vbd_kernels` — 10.8 s - `vbd.particle_vbd_kernels` — 3 instantiations totalling 13.9 s - `newton.geometry.kernels` — 2 instantiations totalling 11.3 s - Bucket: 24.7 s solver, 11.6 s geometry, 7.9 s viewer, 6.7 s narrowphase

Shared across all three: narrowphase GJK/MPR (~2–8 s/solver), viewer kernels (~3–8 s/solver), and a handful of `newton.geometry.*` modules. These recompile per-scene because they're config-instantiated.

Variance notes

- **NVRTC ComputeCache matters more than the Warp cache.** A "cold" run that wipes only the Warp cache (yesterday's protocol) under-reports cold cost by 20–55 % because the driver still has cubin entries cached. Today's "true cold" Kamino is 132 s vs yesterday's 86 s — same code, same hardware, different cache discipline.
- **First-of-session OS file cache warm-up** can add ~25 % to the first cold compile (e.g., iter 1 MuJoCo at 177 s vs 140 s subsequently).
- **Warm runs are not noise-free** in this protocol — clearing 600 MB of NVRTC cache between cold runs can evict Python pages from OS cache, occasionally inflating the next warm run's import phase by ~30 s. The Example init itself remains clean (~4 s).

Implications

- **Steady-state (any relaunch with cache intact): 5–8 s.** No solver choice penalty at runtime.
- **First-time setup on a fresh machine: ~140 s for rigid robots, ~60 s for cloth.** Plan for it in dev onboarding docs.
- **MuJoCo's compile cost is the most fragmented** — no single hot kernel to target for upstream optimization. Kamino has 2 obvious targets (`llt_blocked`, `fk.kernels`) totalling 44 s if anyone wants to invest there.
- **VBD's lazy first-render compile is a UX trap** — looks "ready" after init but stalls 7 s on first frame. Worth fixing if low-latency first-frame matters.

Bench scripts: `bench_solver_rigorous.sh`, `time_example_phases.py`, `aggregate_rigorous.py`, `parse_kernel_compile.py` (all in repo root).